Ders 23 Çalışma Özeti

Ders 23 Çalışma Özeti

Genel Konular

- Dersin tarihi 23 Aralık 2020 Çarşamba. Ders Zoom üzerinden yapıldı. Dönemin 12. haftası.
  - Ders yaklaşık 00:02'de duyurularla başlar. Konu anlatımı yaklaşık 00:16'da başlar.
  - 00:49 ile 00:58 arasında 10 dakikalık ara var.
  - Hoca konuları tablet üzerinde el yazısıyla anlatıyor.
- Duyurular
  - Vize sonuçları en geç 13. hafta içinde açıklanacak.
- Geri izleme (backtracking)
  - Labirent örneği, çıkmaz sokak (dead end) ve geri dönme.
  - Derinlik öncelikli arama (DFS) ve budama (pruning).
  - Kapsamlı arama (exhaustive search) ile karşılaştırma.
  - Durum uzayı ağacı (state space tree), umut veren (promising) ve umut vermeyen (non-promising) düğümler.
- Alt küme toplamı problemi (sum of subsets)
- N-vezir problemi (N-queens)
  - Kaba kuvvet seçenekleri ve geri izlemeyle karşılaştırma.
  - 4-vezir için durum uzayı ağacı.
  - Çapraz tehdit kontrolü ve tek boyutlu dizi gösterimi.
- Genel geri izleme sözde kodu ve karmaşıklık analizinin zorluğu.
- Dal ve sınır (branch and bound)
  - Optimizasyon problemleri, uygun çözüm (feasible solution) ve en iyi çözüm (optimal solution).
  - Atama problemi (assignment problem) ve en iyi öncelikli dal ve sınır (best-first branch and bound).

Hocanın Özellikle Vurguladığı Kısımlar

- Geri izleme, DFS ile budamanın birleşimidir.
  - Umut vermeyen bir düğüme gelince onun altındaki bütün kombinasyonlar elenir.
- Geri izleme her girdide az işlem yapacağını garanti etmez.
  - Girdiye ve verinin dağılımına bağlıdır. Karmaşıklığı genel olarak kesin ifade edilemez.
  - Kapsamlı aramaya göre sonuca makul sürede ulaşma ihtimali çok daha yüksektir.
- Alt küme toplamında sayılar önce küçükten büyüğe sıralanmalıdır.
  - Sıralama, "toplam çok büyük" ve "toplam çok küçük" kontrollerini mümkün kılar.
- N-vezirde konumları $n \times n$ matris yerine n elemanlı bir dizide tutun.
  - Dizinin indisi satırı, değeri sütunu gösterir.
- Çapraz tehdit kontrolü: $|c_i - c_j| = |r_i - r_j|$ ise iki vezir birbirini tehdit eder.
  - Mutlak değer kullanılmazsa öbür çapraz yön kaçırılır.
- Simetri (ayna) kullanılarak N-vezirde arama uzayı yarıya indirilebilir.
- Dal ve sınırda her adımda o an en iyi sınıra sahip düğümden devam edilir. Bu stratejiye en iyi öncelikli (best-first) denir.
- Sınavdaki kod sorularında değerlendirme normal dönemdeki gibidir: çalışmayan ama algoritması kısmen doğru kod da kısmi puan alabilir.

Kısa Tekrar Notları

- **Geri izleme:** Bir yolu dene. Çıkmaz sokaksa bir adım geri dön. Başka seçenek yoksa bir adım daha geri dön.
- **Durum uzayı ağacı:** Her düğüm kısmi bir çözümdür.
  - **Umut veren düğüm:** Altında çözüm olma ihtimali var.
  - **Umut vermeyen düğüm:** Altında çözüm yok, budanır.
- **Alt küme toplamı:** s o anki toplam, d hedef olsun. Düğüm şu durumlarda umut vermez:
  - $s + a_{i+1} > d$ (toplam çok büyük)
  - $s + \sum_{j=i+1}^{n} a_j < d$ (toplam çok küçük)
- **8-vezir, kaba kuvvet seçenekleri:**
  - Her vezir herhangi bir karede: $\binom{64}{8} = 4\,426\,165\,368$
  - Her satırda bir vezir: $8^8 = 16\,777\,216$
  - Her satır ve sütunda bir vezir: $8! = 40\,320$
  - Geri izleme: 2057 düğüm (bütün 92 çözüm için), yaklaşık 20 kat daha az.
- **N-vezir çözümü:** $n = 2$ ve $n = 3$ için çözüm yok. En küçük önemli örnek $n = 4$.
- **Dal ve sınır:** Optimizasyon problemleri için. Minimizasyonda alt sınır (lower bound), maksimizasyonda üst sınır (upper bound) kullanılır.
- **Atama problemi, ilk alt sınır:** Her satırın en küçük değerlerinin toplamı (çakışma olsa bile).

Detaylı Açıklamalar

Duyurular ve dersin işleyişi

- Final dönemi bildiği kadarıyla iki hafta sürecek.
- Hoca derse katılım ve etkileşim konusundaki düşüncelerini uzun anlattı. Kalan haftalarda daha az soru sorarak ilerleyeceğini söyledi.

Geri izleme (backtracking)

- Hedefe giderken her durumdan (state) ilerlenir. Çıkmaz bir noktaya (dead end) gelince bir önceki duruma dönülür.
- Labirent örneği: Duvara çarpınca geri dönülür. Başka seçenek kalmadıysa bir adım daha geri gidilir. Bu şekilde çıkışa ulaşılır.
- Geri izleme, DFS mantığını kullanır ve üstüne budama ekler.
  - Budama: Ağacın bir noktasındaki seçenekler ortadan kaldırılır, aramaya başka daldan devam edilir.
- Kapsamlı arama (exhaustive search) bütün olasılıkları dener.
  - Kombinatoryal problemlerde bu 2^n veya $n!$ gibi sayılara ulaşır ve polinom zamanda bitmez.
  - Geri izleme, çözüme götürmeyecek seçenekleri erkenden eleyerek arama uzayını daraltır.

Alt küme toplamı (sum of subsets)

- Problem: n pozitif tam sayıdan oluşan bir kümede toplamı d olan alt kümeleri bul.
- Örnek 1: $A = \{1, 2, 5, 6, 8\}$, $d = 9$.
  - Çözümler: $\{1, 2, 6\}$ ve $\{1, 8\}$.
  - Kaba kuvvetle $2^5 - 1 = 31$ boş olmayan alt kümenin denenmesi gerekir.
- Örnek 2: $S = \{3, 5, 6, 7\}$, $d = 15$. Durum uzayı ağacı şöyle kuruldu:
  - Kök: toplam 0. Her seviyede bir sayı ya alınır ya alınmaz.
  - 3 alındı (3), 5 alındı (8), 6 alındı (14). $14 + 7 > 15$ olduğu için bu dal umut vermez.
  - 3, 5 alındı, 6 alınmadı (8). 7 alındı: $8 + 7 = 15$. **Çözüm:** $\{3, 5, 7\}$**.**
  - 3 alındı, 5 alınmadı, 6 alındı (9). $9 + 7 = 16 > 15$, umut vermez.
  - 3 alındı, 5 ve 6 alınmadı (3). $3 + 7 = 10 < 15$, umut vermez.
  - 3 alınmadı, 5 alındı, 6 alındı (11). $11 + 7 > 15$, umut vermez.
  - 3 alınmadı, 5 alındı, 6 alınmadı (5). $5 + 7 < 15$, umut vermez.
  - 3 ve 5 alınmadı (0). Kalanların toplamı $6 + 7 = 13 < 15$, umut vermez.

- Python ile kontrol: Bu kümede toplamı 15 olan tek alt küme $\{3, 5, 7\}$.

N-vezir problemi (N-queens)

- Vezirler birbirini yemeyecek şekilde tahtaya yerleştirilir. Vezir yatay, dikey ve çapraz hareket eder.
- 8-vezir için seçenek sayıları "Kısa Tekrar Notları"ndaki gibidir. Kontrol eden bilgiler eklendikçe seçenek sayısı azalır.
- $n = 2$ ve $n = 3$ için çözüm yoktur. En küçük önemli örnek 4×4 tahtadır.
- 4-vezirin durum uzayı ağacı (satır, sütun 0'dan sayılır):
  - İlk vezir (0,0): İkinci vezir yalnızca (1,2) veya (1,3) olabilir.
    - (1,2) seçilirse üçüncü satıra hiç vezir konamaz. Budanır.
    - (1,3) seçilirse üçüncü vezir (2,1) olur. Dördüncü satıra vezir konamaz. Budanır.
  - İlk vezir (0,1): İkinci vezir yalnızca (1,3) olabilir. Üçüncü vezir (2,0), dördüncü vezir (3,2).
  - **İlk çözüm: (0,1), (1,3), (2,0), (3,2).** Dizi gösterimiyle $[1, 3, 0, 2]$.
- Çapraz tehdit kontrolü: Vezir i, (r_i, c_i); vezir j, (r_j, c_j) konumunda olsun.
  - $|c_i - c_j| = |r_i - r_j|$ ise aynı çaprazdadırlar.
  - Aynı sütun kontrolü: $c_i = c_j$.
  - Her satıra tek vezir konduğu için satır kontrolüne gerek yoktur.
- Konumlar n elemanlı bir dizide tutulur. Örnek: $[2, 3, -, -]$ dizisi, 0. satırda 2. sütunda ve 1. satırda 3. sütunda bir vezir olduğunu gösterir. Bellek n^2 yerine n olur.
- Simetri: Tahtanın ortasından ayna çekildiğinde yansıyan yerleşimler aynı sonucu verir. Bu yüzden ilk satırda yalnızca yarı sütunların denenmesi yeterlidir.

```c
/* (row, col) konumuna vezir konabilir mi? col[0..row-1] önceki vezirler */
int promising(int row, int col, int cols[]) {
    for (int i = 0; i < row; i++)
        if (cols[i] == col || abs(cols[i] - col) == row - i)
            return 0;
    return 1;
}
```

Genel geri izleme sözde kodu

- Hoca genel bir sözde kod yazdı. Mantığı şöyle:
  - i. seviyede bir çözüm bulunduysa yazdır.
  - Bulunmadıysa kısıtları bozmayan bir sonraki elemanı ekle ve yeniden çağır.
  - Hiçbir eleman kısıtları sağlamıyorsa geri dön.
- Bütün çözümler isteniyorsa ağacın tamamı, budanan dallar hariç, gezilmelidir. Tek çözüm isteniyorsa ilk çözümde durulabilir.

```
Backtrack(X[1..i]):
    if X[1..i] bir çözüm ise: yazdır
    else:
        for her x in S(i+1) (kısıtlara uyan aday):
            X[i+1] = x
            Backtrack(X[1..i+1])
```

- Karmaşıklık: Birinci seviyede c_1, ikincide c_2 çocuk varsa düğüm sayısı $1 + c_1 + c_1 c_2 + \ldots$ biçiminde büyür. Ama budamanın nerede olacağı girdiye bağlıdır. Bu yüzden genel ve kesin bir karmaşıklık verilemez.
- Uygulama alanları: sudoku, ders programı yerleştirme, birbirini engellememesi gereken her türlü yerleştirme problemi.

Dal ve sınır (branch and bound)

- Optimizasyon problemleri için kullanılır. Amaç bir amaç fonksiyonunu (objective function) kısıtlar altında en büyük veya en küçük yapmaktır.
  - **Uygun çözüm (feasible):** Bütün kısıtları sağlayan çözüm.

  - **En iyi çözüm (optimal):** Uygun çözümler arasında amaç fonksiyonu en iyi olan.
- Her düğümde, o düğümün altında ulaşılabilecek en iyi değer için bir sınır hesaplanır. Sınırı mevcut en iyi çözümden kötü olan dallar budanır.

Atama problemi (assignment problem)

- 4 kişi ve 4 iş var. Her kişiye bir iş atanacak, toplam maliyet en az olacak. Maliyet matrisi kareden alındı:

	İş 1	İş 2	İş 3	İş 4
a	9	2	7	8
b	6	4	3	7
c	5	8	1	8
d	7	6	9	4

- **Kök:** Her satırın en küçüğü alınır: $2+3+1+4=10$. Bu bir alt sınırdır. Bu atama geçerli değildir, çünkü b ve c aynı işe (iş 3) düşer.
- **1. seviye (a'nın işi):** a'ya bir iş verilir. Diğer satırlarda o sütun hariç en küçükler toplanır.
  - a→1: $9+3+1+4=17$
  - a→2: $2+3+1+4=10$
  - a→3: $7+4+5+4=20$
  - a→4: $8+3+1+6=18$
  - En küçük sınır a→2 (10). Oradan devam edilir.
- **2. seviye (a→2 altında b'nin işi):**
  - b→1: $2+6+1+4=13$
  - b→3: $2+3+5+4=14$
  - b→4: $2+7+1+7=17$
  - En küçük sınır b→1 (13).
- **3. seviye:** c→3, d→4 ise maliyet $2+6+1+4=13$. c→4, d→3 ise $2+6+8+9=25$.
- **Sonuç:** a→2, b→1, c→3, d→4. Toplam maliyet 13. Sınırı 13'ten büyük olan diğer bütün düğümler (14, 17, 17, 18, 20) budanır.
- Python ile bütün $4!=24$ atama denendi. En küçük maliyet 13 ve atama aynı.
- Bir öğrenci, b→1'de sınır 13 bulununca altına bakmaya gerek olup olmadığını sordu. Hoca, sınırın gerçekleşmeyebileceğini, iki kombinasyonun da kontrol edilmesi gerektiğini söyledi.

Ders sonrası notlar

- **"1 vezir problemi çözümsüz" ifadesi:** Hoca 1×1 tahtada çözüm olmadığını söyledi. Doğrusu: $n=1$ için tek vezirin tek kareye konması bir çözümdür. Çözümü olmayanlar $n=2$ ve $n=3$'tür.
- **"Backtracking N-vezir problemini doğrusal zamanda çözer" ifadesi:** Bu doğru değil. Geri izleme yalnızca kapsamlı aramaya göre denenen düğüm sayısını azaltır. En kötü durumda yine üstel çalışabilir.
- **"Bunlar NP problemlerdir ve polinom zamanda çözülemez" ifadesi:** Bir problemin NP'de olması onun polinom zamanda çözülemeyeceğini kanıtlamaz; P ile NP'nin eşit olup olmadığı açık bir sorudur. Ayrıca N-vezirin tek bir çözümünü bulmak için polinom zamanlı açık kurgular bilinmektedir. Zor olan bütün çözümleri saymak ya da büyük olasılıkla bazı kısıtlı sürümleridir.
- **C→4, D→3 yaprağı:** Hoca 25'i bulduktan sonra bunun "geçerli olmayan bir sonuç" olduğunu söyledi. Doğrusu bu atama da geçerli (uygun) bir atamadır; yalnız 13'ten pahalı olduğu için en iyi değildir.
- **2057 düğüm:** Python ile doğrulandı. 8-vezirin bütün 92 çözümünü bulmak için geri izleme ağacında 2057 umut veren düğüm ziyaret edilir. İlk çözüm 114 düğümde bulunur. 4-vezirde bütün çözümler (2 tane) için 17 düğüm gezilir.
- **"Branch and bound maksimizasyon problemleri için":** Dal ve sınır genel olarak optimizasyon problemleri içindir. Bu derste çözülen atama problemi zaten bir minimizasyon problemidir.

- **Beam search sorusu:** Bir öğrenci beam search ile benzerliği sordu, hoca kesin bilgi vermedi. Kısa cevap: Beam search de bir ağaç araması yapar, ama her seviyede yalnızca en iyi k düğümü tutar ve en iyi çözümü garanti etmez. Dal ve sınır ise sınırlara dayanarak budama yapar ve en iyi çözümü garanti eder.
- **Matristen gelen sezgisel çözüm:** Kökteki 10 değeri gerçek bir atama değildir. Yalnızca bir alt sınırdır. Gerçek en iyi değer 13 çıkar.
- **Altyazı hataları:** "Calum/Kalım" column, "Rho/rov" row, "coin" queen, "Bruning" pruning, "esayen/eshan" assign demektir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.